

DMIT2015 — Lesson 4A Student Practice Activity

Java Review + Unit Testing Reinforcement Overview

In this activity, you will:

- Create a Java class using IntelliJ IDEA
- Add constructor validation
- Create JUnit test methods
- Use AssertJ assertions
- Run and verify automated tests

This activity reinforces:

- Java syntax
- packages and classes
- IntelliJ workflow
- JUnit testing
- AssertJ assertions

Learning Outcomes

By completing this activity, you should be able to:

- Create Java classes in a package
- Write constructor validation logic
- Create and run JUnit tests
- Use AssertJ assertions
- Apply Arrange–Act–Assert structure

Part 1 — Create the Product Class

Step 1 — Create Package

Create the following package:

```
dmit2015.model
```

Step 2 — Create Product.java

Create the following Java class:

```
package dmit2015.model;

public class Product {

    private String name;
    private double price;

    public Product(String name, double price) {
```

```

        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Product name is required");
        }

        if (price <= 0) {
            throw new IllegalArgumentException("Price must be greater than 0");
        }

        this.name = name;
        this.price = price;
    }

    public String getName() {
        return name;
    }

    public double getPrice() {
        return price;
    }

    public double getPriceWithTax() {
        return price * 1.05;
    }
}

```

Part 2 — Create ProductTest.java

Step 1 — Create Test Class

Create the following package if it does not already exist:

```
src/test/java/dmit2015/model
```

Create:

```
ProductTest.java
```

Step 2 — Add Required Imports

```

import org.junit.jupiter.api.Test;

import static org.assertj.core.api.Assertions.*;
import static org.assertj.core.api.Assertions.within;

```

Part 3 — Create Test Methods

Test 1 — Verify Product Name

```

@Test
void getName_whenProductCreated_shouldReturnCorrectName() {

    // Arrange
    Product currentProduct = new Product("Laptop", 1200);

    // Act
    String actualName = currentProduct.getName();
}

```

```
// Assert
assertThat(actualName)
    .isEqualTo("Laptop");
}
```

Test 2 – Verify Product Price

```
@Test
void getPrice_whenProductCreated_shouldReturnCorrectPrice() {

    // Arrange
    Product currentProduct = new Product("Laptop", 1200);

    // Act
    double actualPrice = currentProduct.getPrice();

    // Assert
    assertThat(actualPrice)
        .isCloseTo(1200, within(0.01));
}
```

Test 3 – Verify Tax Calculation

```
@Test
void getPriceWithTax_whenCalled_shouldReturnCorrectValue() {

    // Arrange
    Product currentProduct = new Product("Laptop", 1200);

    // Act
    double actualPriceWithTax = currentProduct.getPriceWithTax();

    // Assert
    assertThat(actualPriceWithTax)
        .isCloseTo(1260, within(0.01));
}
```

Test 4 – Verify Invalid Product Name

```
@Test
void constructor_whenNameIsBlank_shouldThrowException() {

    assertThatThrownBy(() ->
        new Product("", 100))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessageContaining("required");
}
```

Test 5 – Verify Invalid Price

```
@Test
void constructor_whenPriceIsNegative_shouldThrowException() {

    assertThatThrownBy(() ->
        new Product("Laptop", -5))
        .isInstanceOf(IllegalArgumentException.class)
        .hasMessageContaining("greater than 0");
}
```

Part 4 — Run the Tests

Instructions

Run:

`ProductTest.java`

Verify:

- All tests pass
- IntelliJ displays a green test result bar

Bonus Challenge (Optional)

Add:

```
private int quantity;
```

Then:

- update constructor
- create getter
- create:

```
getInventoryValue()
```

Formula:

```
price * quantity
```

Create a corresponding unit test.

Submission

No Brightspace submission required unless instructed by your instructor.

Be prepared to:

- demonstrate your code
- explain your tests
- show successful test execution